Case study - Modelling and validation of 2nd order ECM and EKF for SOC estimation

---

Task 1: Lookup Table Implementation

Objective - : Develop lookup tables for battery OCV and ECM parameters as functions of SOC and C-rate for using in a Simulink battery model.

OCV- SOC lookup table

- Data file: 1-A123 Aging Tests_Data Summary.xlsx (206k 0km case)
- Build a 1D lookup table based on SOC

R0, R1, C1, R1 & C2 look up table

- Data file: ecm_lookup_table.csv
- This is a 2D lookup table where each parameter is based on SOC and C rate.

Data Preprocessing

The raw data contained non-uniform C-rate breakpoints across SOC, requiring preprocessing for Simulink compatibility.

Preprocessing using simulink script

1. Load the datafile into matlab. The file contains the columns SOC, Crate, R0, R1, C1, R2, C2
2. Extract common independent variables (SOC and Crate) and dependent variables (R0, R1, C1, R2, C2)
3. Defined common breakpoints for both the variables
    a. SOC: 0,5,10, …….100
    b. C rate: [0.25 0.5 1 2 5 8 10 15]
4. Create the rectangular grid
5. Create separate data tables for each parameter where it is taking the scattered datapoints from independent and dependent variables and Interpolates them onto a regular SOC-C rate grid using linear interpolation.
6. Save all the data tables into Battery_LUTs.mat

Output

Parameter	Lookup Table Type	Independent Variables
OCV	1D	SOC
R0	2D	SOC, C-rate
R1	2D	SOC, C-rate
R2	2D	SOC, C-rate

C1	2D	SOC, C-rate
C2	2D	SOC, C-rate

Task 2: ECM (2nd Order) Implementation

Objective - : Implement a second-order equivalent circuit model (2RC ECM) in Simulink to estimate battery terminal voltage using SOC-dependent OCV and SOC/C-rate dependent ECM parameters.

Coulomb counting for soc estimation

A subsystem is implemented with the following equations

$$SOC(t) = SOC(0) + \int \frac{Idt}{Q}$$

Unit conversion of Ah to As is done by multiplying the Q by 3600s

C rate calculation

$$C\,rate = |I|/Q$$

An Abs operation block is used to get the Modulus.

Inputs:

- Initial soc (0-1)
- Current (A)
- Capacity (Ah)

Outputs:

- SOC (%)
- C rate

Terminal Voltage estimation (2RC ECM)

ECM Voltage Equation

$$V(t) = OCV(SOC(t) + I(t)R0 + V_{RC1}(t) + V_{RC2}(t)$$

RC Branch Dynamics

$$V_{RC1}(t) = e^{-\frac{\Delta t}{R1C1}} V_{RC1}(t-1) + R1(1 - e^{-\frac{\Delta t}{R1C1}})I(t-1)$$

$$V_{RC2}(t) = e^{-\frac{\Delta t}{R2C2}} V_{RC2}(t-1) + R2(1 - e^{-\frac{\Delta t}{R2C2}})I(t-1)$$

The above equations are implemented using simulink blocks. In order to not get an algebraic loop error, a unit delay block is placed where the $V_{RC1}(t-1)$ and $V_{RC2}(t-1)$ are taken back to the equation.

Inputs:

- Current (A)
- OCV(V)
- R0 (Ω)
- R1(Ω)
- C1(F)
- R2(Ω)
- C2(F)

Outputs:

- Simulated terminal voltage (V)

Task 3: ECM Validation

Objective -: Validate the implemented 2nd-order ECM by comparing simulated terminal voltage with measured voltage under a real-world drive cycle.

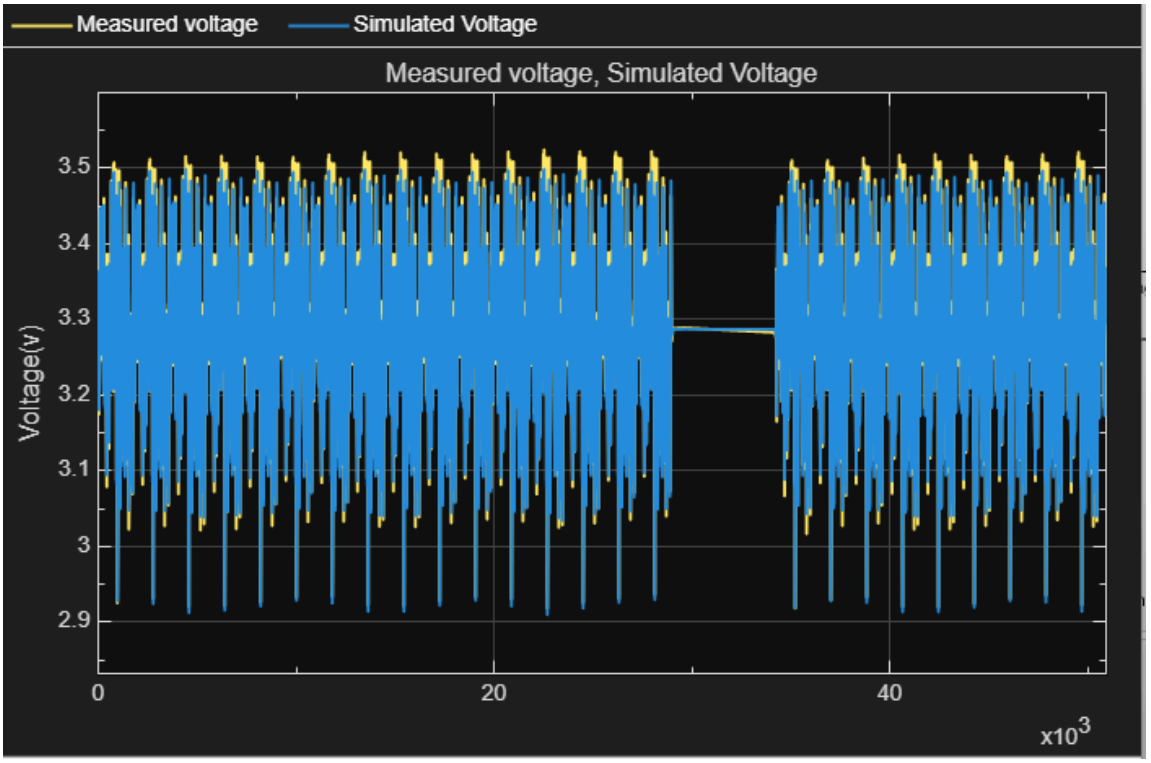
Test profile chosen for validation: WLTP profile (Data file:10-24-19_16.28 960_WLTP206a.mat)
Steps:

- Loaded the WLTP dataset in MATLAB.
- Extracted: I_ts → Current time series, V_ts → Measured voltage time series.
- Used I_ts as the current input for the ECM model.
- Simulated terminal voltage and compared it with V_ts.

Simulation settings

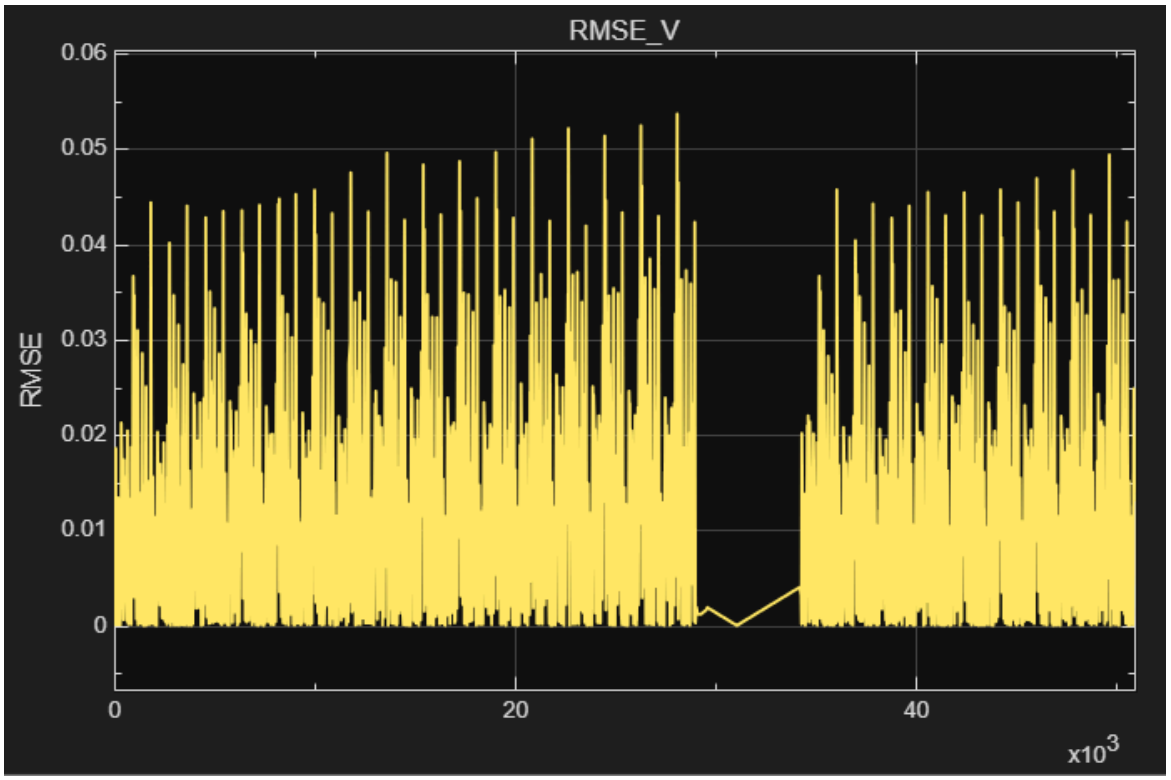
- Step size: 1 s (matching dataset sampling time)
- RMS error calculated using RMS measurement block.

Plots



Graph: actual vs simulated voltage graph

The yellow curve corresponds to the measured data, while the blue curve represents the simulated voltage.



Graph: RMS value during the test

Observations

- Voltage discrepancies are mainly observed at peak regions.
- These peaks correspond to high current conditions (~15C).
- High C-rate regions are not covered in the available parameter dataset.
- The model therefore relies on linear extrapolation, leading to increased error at peaks.
- Max RMSE value - 0.0537

Conclusion

The ECM demonstrates good overall accuracy under the WLTP profile, with low RMS error. Deviations at high current peaks highlight the need for parameter data at higher C-rates to further improve model fidelity.

Task 4: Extended Kalman Filter (EKF) Implementation

Objective

Implement an Extended Kalman Filter (EKF) based on a nonlinear 2RC equivalent circuit model to estimate battery State of Charge (SOC) in a discrete-time framework using measured terminal voltage and current.

The Extended Kalman Filter (EKF) was implemented from scratch to estimate the battery State of Charge (SOC) using a nonlinear equivalent circuit model (ECM). Both prediction and correction stages were explicitly formulated using matrix operations, and the filter was implemented in a discrete-time framework.

Battery State-Space Model

The nonlinear battery model is represented in state-space form as:

$$x(k) = A\,x(k) + B\,u(k)$$
$$y(k) = g(x(k), u(k))$$

where the state vector is defined as:

$$x(k) = [SoC(k) \quad VRC1(k) \quad VRC2(k)]^T$$

definitions,

- SOC(k): State of charge (0–1)
- *VRC1(k)*: Voltage across RC pair 1 (V)
- VRC2(k): Voltage across RC pair 2 (V)
- I(k): Battery current (A)
- V(k): Terminal voltage (V)

The system matrices A and B are derived from the 2RC equivalent circuit model combined with Coulomb counting equations.

System Matrices

The discrete-time state transition and input matrices are given by:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & e^{-\Delta t/\tau_{RC1}} & 0 \\ 0 & 0 & e^{-\Delta t/\tau_{RC2}} \end{bmatrix} \qquad B = \begin{bmatrix} \dfrac{\Delta t}{Q_{Nom}} \\ R_1\left(1 - e^{-\frac{\Delta t}{\tau_{RC1}}}\right) \\ R_2\left(1 - e^{-\frac{\Delta t}{\tau_{RC2}}}\right) \end{bmatrix}$$

where R1,C1,R2,C2 are ECM parameters and Δt is the sampling interval. These parameters are updated at each time step, making the matrices time-varying.

Noise and Covariance Matrices

- **Process Noise Covariance (Q)** Models uncertainties in the system dynamics due to parameter variation, aging effects, temperature dependency, and modeling inaccuracies.
- **Measurement Noise Covariance (R)** Represents uncertainty in terminal voltage measurements, including sensor noise, quantization errors, and electrical disturbances.
- **Error Covariance Matrix (P)** Represents uncertainty in the estimated states. Larger values indicate lower confidence in the estimate.

EKF Algorithm Steps

Step 1: Initialization

The EKF is initialized with:

- Initial state estimate x(0).
- Initial error covariance P(0).
- Process noise covariance Q
- Measurement noise covariance R

Step 2: Prediction Step

The predicted state and error covariance are computed as:

$$\widehat{x}(\ k)\ =\ Ax(k\ -\ 1)\ +\ B\,u(k\ -\ 1)$$

$$P(k)\ =\ A\,P(k\ -\ 1)AT\ +\ Q$$

This step predicts the system behavior based solely on the model.

Step 3: Measurement Linearization

Since the measurement equation is nonlinear, it is linearized about the predicted state using the Jacobian matrix C(k). The Jacobian is obtained from the slope of the OCV–SOC characteristic curve. A lookup table (LUT), generated using MATLAB, is used to obtain this slope during simulation.

Step 4: Correction Step

The Kalman gain is calculated as:

$$L(k)\ =\ P^{-}(k)\,C(k)^{\mathrm{T}}\,(\,C(k)\,P^{-}(k)\,C(k)^{\mathrm{T}}\ +\ R\,)^{-1}$$

The state and error covariance are then updated:

$$\hat{x}(k)\ =\ \hat{x}^{-}(k)\ +\ L(k)[V\_exp(k)\ -\ g(\hat{x}^{-}(k),\ u(k))]$$

$$P(k)\ =\ (I\ -\ L(k)C(k))P^{-}(k)$$

EKF Simulation Implementation

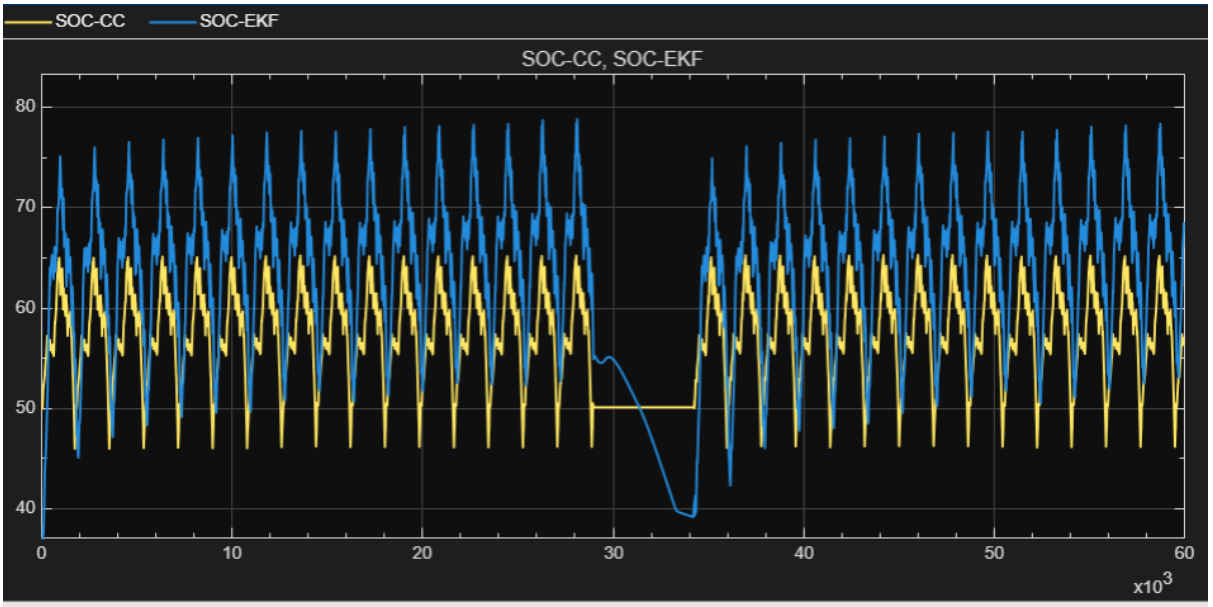
- The EKF was implemented in a discrete-time simulation framework, where all states and parameters are updated at each sampling instant. At every time step, the battery ECM parameters (R1,C1,R2,C2) are updated based on the current SOC estimate, and the corresponding system matrices A(k) and B(k) are recalculated.
- The prediction step propagates the state and error covariance using the updated system model and the input current from the drive cycle. The nonlinear measurement equation is linearized using the Jacobian C(k), obtained from the slope of the OCV–SOC characteristic curve via a lookup table (LUT).
- Measured terminal voltage data is then incorporated during the correction step to compute the Kalman gain and update the state estimate. The corrected SOC value, obtained from the state vector, is logged at each time step for validation against Coulomb counting. This recursive process is repeated for the entire simulation duration, ensuring time-synchronized state estimation and suitability for real-time and embedded implementation.

Task 5: EKF Validation & Tuning

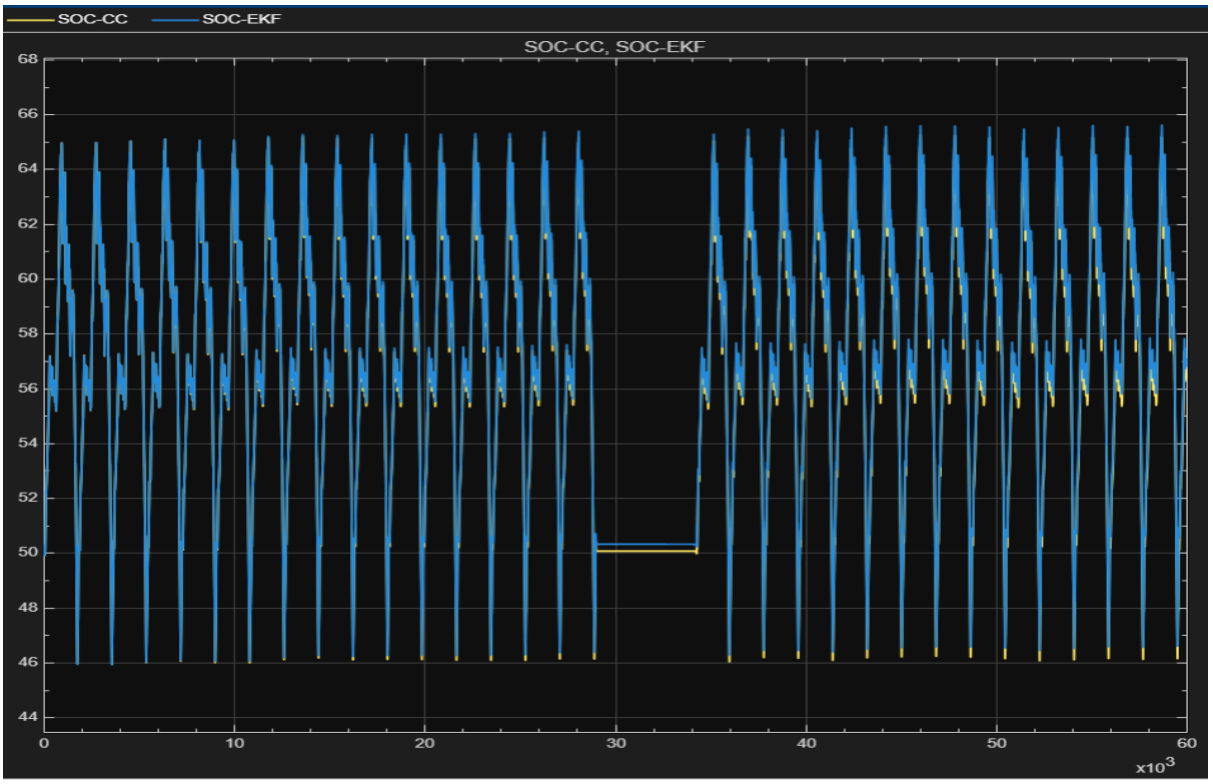
Objective:

To validate and tune the EKF-based SOC estimation by aligning the predicted SOC with Coulomb counting SOC through adjustment of the Q and R matrices and accurate initial SOC initialization.

Before tuning the system, while comparing the EKF predicted SOC with the Coulomb count SOC, the graph is as follows;



The tuning is basically done to the Q matrix which accounts for the model errors. Decreased the value and can observe the EKF more aligned to the actual curve. And changing the initial value of SOC considered in EKF to the actual initial SOC made the EKF SOC curve aligned to the actual SOC curve.



The tuning was trial and error method and found the pattern of the predicted value for each value and matched the output by tuning the parameters in the Q matrix and the R value.

Max RMSE value - 0.63